R Data Wrangling Reference

Notes from cleaning and analyzing the Kia/Hyundai break-in dataset (DATA 607)

This guide walks through the full workflow in order: reshaping a messy CSV, cleaning column names and strings, fixing type and date problems, summarizing, visualizing many categories, grouping cities into regions, and computing month-over-month change. Each section includes the working code and the common errors hit along the way.

1. Transposing a CSV

Base R uses `t()`, which returns a matrix and coerces every column to one type (usually character). Convert back to a data frame and restore types afterward.

```
df <- read.csv("data.csv")
df_t <- as.data.frame(t(df[, -1]))
colnames(df_t) <- df[, 1]
df_t <- cbind(variable = rownames(df_t), df_t)
rownames(df_t) <- NULL
df_t <- type.convert(df_t, as.is = TRUE)   # restore numeric columns
```

Tidyverse transposes by pivoting twice, which keeps types cleaner:

```
df_t <- df |>
  pivot_longer(-1, names_to = "variable") |>
  pivot_wider(names_from = 1, values_from = value)
```

One-liner: `data.table::transpose(df, keep.names = "variable", make.names = 1)`

What pivot_longer(-1) means

`-1` selects every column *except* column 1. Column 1 stays as the identifier and is repeated on each row; the other column names go into `name` (or whatever `names_to` says) and their contents go into `value`. Selecting by name, e.g. `pivot_longer(-location)`, is safer because it won't break if the column order changes.

2. Fixing column-name problems

Error: subscript can't contain the empty string

`pivot_wider()` failed because a value in the column becoming the new headers was `""`. A column can't be named with an empty string. Check with `df[[1]]`, then either drop or fill the blank:

```
df_t <- df |>
  rename(id = 1) |>                                  # rename the column itself
  mutate(across(1, ~ if_else(is.na(.x) | .x == "",   # replace blank VALUES
                             "unnamed", .x))) |>
  pivot_longer(-1, names_to = "variable") |>
  pivot_wider(names_from = 1, values_from = value)
```

- **mutate()** changes cell values; **rename()** changes column names. They are separate steps.
- `across(1, ~ ...)` applies a function to column 1; ~ creates an anonymous function and `.x` is the column.
- If the real headers are in row 1, re-read with `read_csv("data.csv", skip = 1)`.
- If pivot_longer() complains about combining double and character, add `values_transform = as.character` and run `type_convert()` afterward.

Filling X.1, X.2 placeholder names

`read.csv()` auto-names blank header cells `X.1`, `X.2`, etc. When a location header spans three columns, replace the placeholders with NA and fill downward:

```
df_t <- df_t |>
  mutate(variable = if_else(str_detect(variable, "^X\\.\\d+$"), NA, variable)) |>
  fill(variable, .direction = "down") |>
  group_by(variable) |>
  mutate(rep = row_number()) |>     # optional: keep the 3 rows distinguishable
  ungroup()
```

Prevention: `read.csv()` also turns `"Amarillo, TX"` into `Amarillo..TX`. Use `read_csv()` or `read.csv(check.names = FALSE)` to keep the original header text.

3. Long vs. wide format

Long format is usually *better* for tidyverse work: `group_by()`, `summarise()`, `filter()`, and `ggplot()` all expect one value column with descriptors in their own columns.

It gets awkward when you need to combine **different measures for the same observation** (e.g. Kia count divided by total), because those numbers sit in different rows. Pivot wider just enough to put them side by side, then compute.

Tidy data rule of thumb: each variable gets its own column, each observation its own row. Cities, years, and categories are usually values (keep long). Things measured in different units or meaning different things are separate variables (keep as columns). If you pivot wider before every mutate(), some of your “values” are really variables.

4. String cleanup

Removing part of a string

```
mutate(x = str_remove(x, "abc"))            # first occurrence
mutate(x = str_remove_all(x, "abc"))        # every occurrence
mutate(x = str_remove_all(x, fixed(".")))   # literal dot (. is a regex wildcard)
mutate(x = str_remove(x, "^X"))             # only at the start
mutate(x = str_remove(x, "_2024$"))         # only at the end
```

Base R: `gsub("abc", "", df$x, fixed = TRUE)`. To *keep* part of a string, use `str_extract()` instead.

Values that look identical but group separately

Two “All” groups usually means invisible differences: trailing spaces, non-breaking spaces (common from web/Excel data), or capitalization.

```
df |> count(col)                                # see the groups
unique(df$col) |> nchar()                       # "All" should be 3
unique(df$col) |> sprintf(fmt = "[%s]")         # brackets reveal spaces
charToRaw(unique(df$col)[2])                    # 20 = space, c2 a0 = non-breaking

df <- df |>
  mutate(col = col |>
           str_replace_all("\u00a0", " ") |>    # non-breaking -> regular space
           str_squish())                         # trim ends + collapse inner spaces
```

5. Summarizing and the errors along the way

Error: invalid 'type' (character) of argument

Double quotes make a literal string, so `sum("All")` tries to add up the word itself. The fix depends on the data shape:

```
# Column names with special characters -> backticks
summarize(prop = sum(`Kia/Hyundais`) / sum(All))

# Categories stored as values in one column -> subset inside sum()
summarize(prop = sum(value[category == "Kia/Hyundais"]) /
                 sum(value[category == "All"]))
```

If `class(df$value)` is `"character"`, convert first with `parse_number()`, which also strips commas, %, and $.

Getting NA for every group

`sum()` returns NA if any input is NA, so add `na.rm = TRUE`. But check the cause:

Likely cause	How to spot it	Fix
pivot_wider() didn't align rows	Staircase pattern: one value per row, rest NA	`pivot_wider(id_cols = Date, ...)`
Values still character	class() says character; odd strings like "1,234"	`parse_number(value)`
Uncleaned duplicate category	Extra column like "All "	`str_squish()` before widening
Dates failed to parse	An NA group in Date	Check `sum(is.na(Date))`

```
df_wide |> head(10)
names(df_wide)
colSums(is.na(df_wide))
```

6. Converting MMM/YY text to dates

```
df <- df |> mutate(Date = my(Date))     # lubridate: "Apr/20" -> 2020-04-01

# Base R alternative
df <- df |> mutate(Date = as.Date(paste0("01/", Date), format = "%d/%b/%y"))
```

- Monthly data gets the 1st of the month by convention.
- Use **Date**, not datetime (POSIXct), for monthly data: no time zone issues in plots.
- Two-digit years: 00–68 become 2000s, 69–99 become 1900s.
- Unparsed values become NA; confirm with `sum(is.na(df$Date))`.
- Display without changing the data: `format(Date, "%b %Y")` or `scale_x_date(date_labels = "%b %Y")`.

7. Plotting

Starting the plot at a later date

```
df |> filter(Date >= as.Date("2021-01-01")) |> ggplot(aes(Date, y)) + geom_line()

# Keep the data, change only the view
+ coord_cartesian(xlim = as.Date(c("2021-01-01", NA)))
```

Filtering is usually best: explicit, no warnings, and smoothing layers use only the shown dates. `scale_x_date(limits = ...)` drops out-of-range points with a “Removed rows” warning.

Too many categories (the plot became all legend)

About 90 cities mapped to color crowded out the plot. Options:

- **Facets:** `facet_wrap(~ location, ncol = 8)`, saved large with `ggsave(..., width = 16, height = 12)`.
- **Highlight:** all lines in gray, a few named cities in color.
- **Hide legend:** `theme(legend.position = "none")` (quick, but lines become unidentifiable).
- **Heatmap:** `geom_tile()` with cities on the y-axis handles many categories well.

Check the data before choosing a chart

Lines pinned to the top of the axis and dropping straight to zero signal infinite or >1 proportions, typically from dividing by a zero total or a misaligned pivot:

```
df |> filter(!is.finite(KiaHyundaiProportion) | KiaHyundaiProportion > 1)
```

Better ways to show the trend

The data shows proportions low and steady until mid-2022, then rising sharply in most cities, consistent with the “Kia Boys” theft trend that spread that summer.

```
# Median city with a band for the middle 50% of cities
df |>
  group_by(Date) |>
  summarize(median = median(KiaHyundaiProportion, na.rm = TRUE),
            q25 = quantile(KiaHyundaiProportion, 0.25, na.rm = TRUE),
            q75 = quantile(KiaHyundaiProportion, 0.75, na.rm = TRUE)) |>
  ggplot(aes(Date)) +
  geom_ribbon(aes(ymin = q25, ymax = q75), fill = "steelblue", alpha = 0.3) +
  geom_line(aes(y = median), color = "steelblue", linewidth = 1) +
  geom_vline(xintercept = as.Date("2022-07-01"), linetype = "dashed") +
  scale_y_continuous(labels = scales::percent) +
  theme_minimal()

# Before/after "dumbbell" chart per city
df |>
  mutate(period = if_else(Date < as.Date("2022-07-01"), "Before", "After")) |>
  group_by(location, period) |>
  summarize(avg = mean(KiaHyundaiProportion, na.rm = TRUE), .groups = "drop") |>
  pivot_wider(names_from = period, values_from = avg) |>
  ggplot(aes(y = reorder(location, After))) +
  geom_segment(aes(x = Before, xend = After, yend = location), color = "gray70") +
  geom_point(aes(x = Before), color = "gray50") +
  geom_point(aes(x = After), color = "firebrick") +
  scale_x_continuous(labels = scales::percent) +
  theme_minimal()
```

Pairing the median/band chart (overall trend) with the dumbbell chart (which cities were hit hardest) tells the most complete story.

Cleaning city names for labels

```
mutate(location = location |>
         str_replace("\\.\\.", ", ") |>    # ".." back to ", " (do this first)
         str_replace_all("\\.", " "))       # remaining dots back to spaces
```

8. Bucketing cities into Census regions

U.S. Census regions are official, widely recognized, and built into base R via `state.abb` and `state.region`.

Region	Cities identified
West	Anaheim, Bakersfield, Boise, Chandler, Chula Vista, Denver, Eugene, Fremont, Glendale, Henderson, Los Angeles, Modesto, Oxnard, Peoria, Portland, Reno, Riverside County, Sacramento, Salt Lake City, San Bernardino, San Diego, San Francisco, San Jose, Spokane, Stockton, Vancouver
South	Amarillo, Arlington, Atlanta, Austin, Corpus Christi, Dallas, Durham, El Paso, Fort Worth, Frisco, Garland, Houston, Irving, Louisville, Lubbock, McKinney, Miami, Montgomery County, Newport News, Norfolk, Orlando, Plano, Prince George's County, Raleigh, San Antonio, St. Petersburg, Virginia Beach, Washington D.C.
Midwest	Aurora, Chicago, Cincinnati, Lincoln, Madison, Milwaukee, Minneapolis, Omaha, Wichita
Northeast	Buffalo, New Haven, Pittsburgh, Providence, Rochester, Syracuse

```
city_state <- c(
  "Atlanta" = "GA", "Bakersfield" = "CA", "Chicago" = "IL",
  "Cincinnati" = "OH", "Dallas" = "TX", "Denver" = "CO",
  "El Paso" = "TX", "Fort Worth" = "TX", "Irving Texas" = "TX",
  "Louisville" = "KY", "Portland" = "OR", "Sacramento" = "CA",
  "San Antonio" = "TX", "San Diego" = "CA", "San Francisco" = "CA",
  "Virginia Beach" = "VA", "Washington D C" = "DC"
)

df <- df |>
  mutate(
    key    = location |> str_replace_all("[.,]", " ") |> str_squish(),
    state  = coalesce(str_extract(key, "\\b[A-Z]{2}$"), city_state[key]),
    region = as.character(state.region[match(state, state.abb)]),
    region = case_when(
      state == "DC"             ~ "South",
      region == "North Central" ~ "Midwest",
      .default = region
    )
  ) |>
  select(-key)

df |> filter(is.na(region)) |> distinct(location)   # anything unplaced
```

- Regions are lopsided (Northeast has ~6 cities). For finer groups, swap in `state.division` for the 9 Census divisions.
- Portland is assumed to be Portland, OR.
- **Sum counts, then divide** when summarizing by region. Averaging each city's proportion gives a small city the same weight as Los Angeles.

9. Month-over-month change with lag()

Grouping by `Date` and `region` leaves one row per group, so `lag()` has no previous row and returns all NA. Summarize first, then regroup by region alone:

```
df_region <- df |>
  # 1. One row per region per month
  group_by(region, Date) |>
  summarize(KiaHyundaiProportion = sum(`Kia/Hyundai`, na.rm = TRUE) /
                                   sum(All, na.rm = TRUE),
            .groups = "drop") |>
  # 2. Each region as its own time series
  group_by(region) |>
  arrange(Date, .by_group = TRUE) |>
  mutate(change = KiaHyundaiProportion - lag(KiaHyundaiProportion)) |>
  ungroup()
```

- `lag()` uses row order, not dates, so `arrange(Date)` is required.
- The first month in each region is NA by design.
- Missing months make lag() compare to an older row; check with `gap = Date - lag(Date)`.
- Percent change: `KiaHyundaiProportion / lag(KiaHyundaiProportion) - 1`.